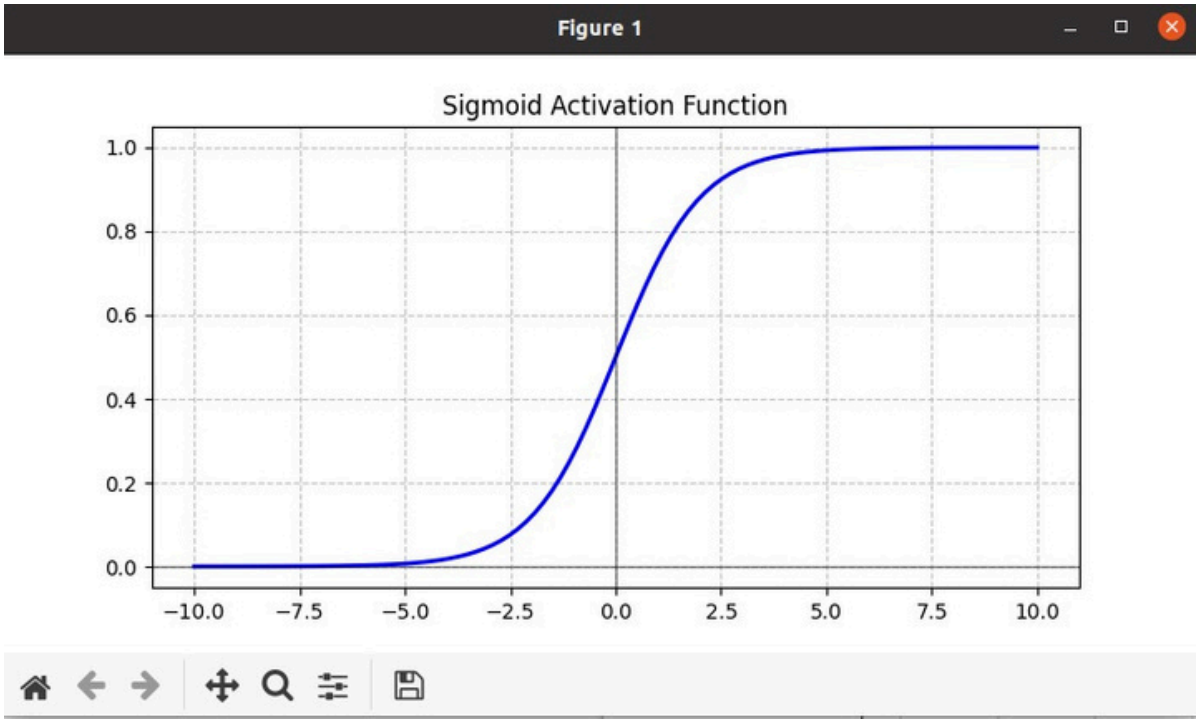
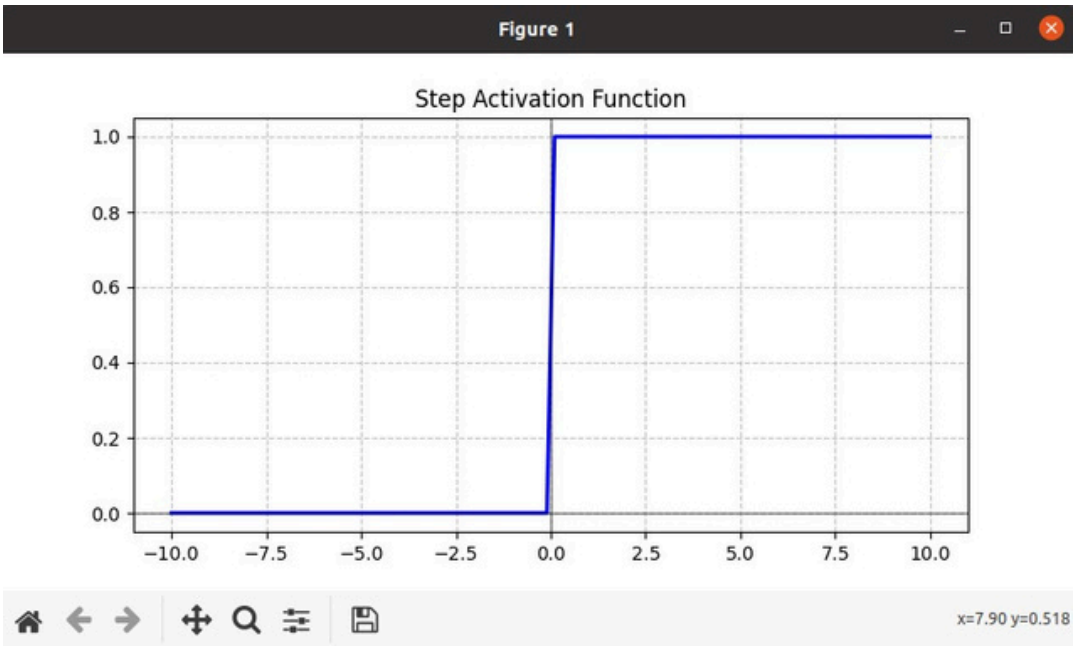
PRACTICAL 1

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-10, 10, 100)
sigmoid = 1 / (1 + np.exp(-x))
plt.plot(x, sigmoid)
plt.title("Sigmoid Activation Function")
plt.grid()
plt.show()
```

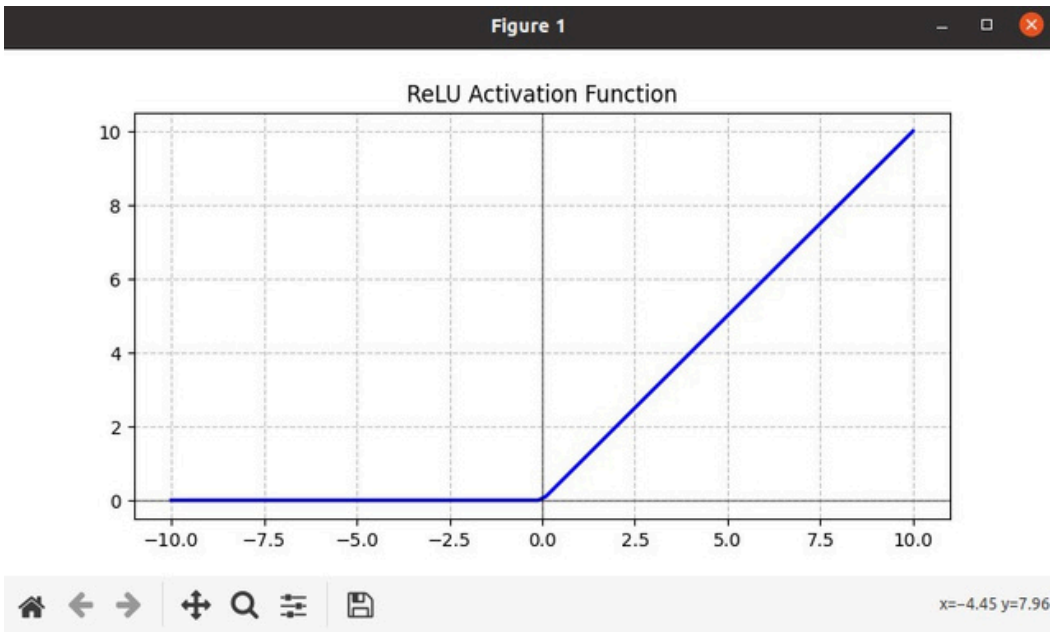



```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-10, 10, 100)
tanh = np.tanh(x)
plt.plot(x, tanh)
plt.title("Step Activation Function")
plt.grid()
```

```
plt.show()
```

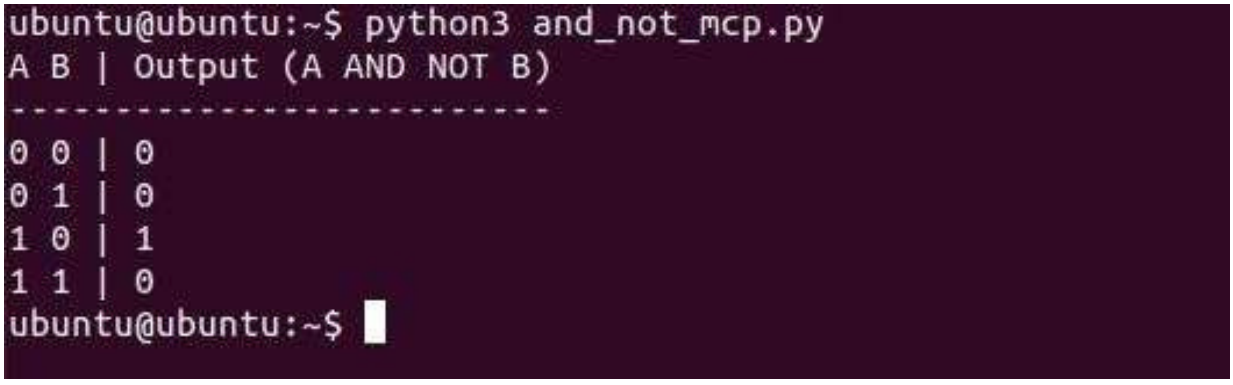



```
import matplotlib.pyplot as plt
x = np.linspace(-10, 10, 100)
relu = np.maximum(0, x)
plt.plot(x, relu)
plt.title("ReLU Activation Function")
plt.grid()
plt.show()
```



PRACTICAL 2

```
# McCulloch-Pitts Neural Network for ANDNOT function
def mcculloch_pitts_andnot(A, B):
    # Assign weights
    w1 = 1    # weight for A
    w2 = -1 # weight for B (NOT B effect)
    # Threshold
    theta = 1
    # Weighted sum
    net_input = (A * w1) + (B * w2)
    # Activation function (step function)
    if net_input >= theta:
        return 1
    else:
        return 0
# Testing all inputs
print("A B | Output")
forAin [0, 1]:
    forB in [0, 1]:
        output = mcculloch_pitts_andnot(A, B)
        print(A, B, "|", output)
```



PRACTICAL-3

```
class     Perceptron:
class Perceptron:
    def init (self, lr=0.1, epochs=10):
        self.lr = lr
        self.epochs =
        epochs self.weight
        = 0
        self.bias = 0

    def activation(self, x):
        return 1 if x >= 0 else 0

    def train(self, X, y):
        for _ in
            range(self.epochs): for
            i in range(len(X)):
                net = X[i] * self.weight +
                self.bias y_pred =
                self.activation(net) error = y[i]
                - y_pred

                # update rule
                self.weight += self.lr * error *
                X[i] self.bias += self.lr * error

    def predict(self, x):
        net = x * self.weight + self.bias
        return self.activation(net)

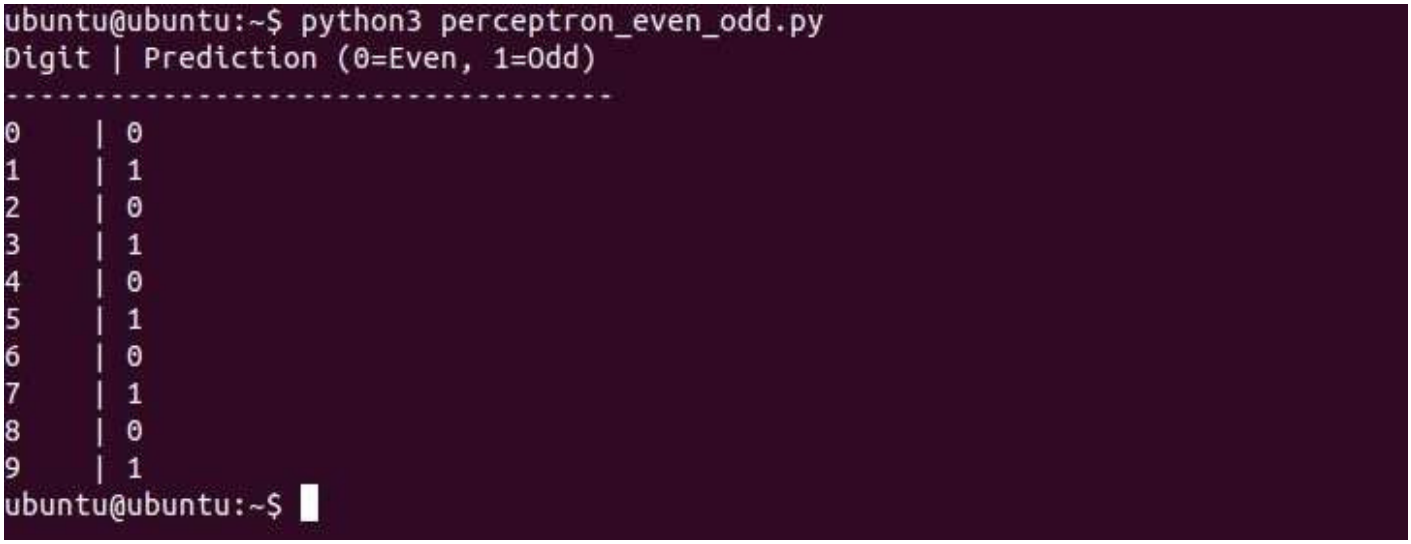

# Training data (digits 0–9)
X = [i % 2 for i in range(10)] # feature: parity (0 or 1) y =
[0 if i % 2 == 0 else 1 for i in range(10)] # labels


# Create and train perceptron
p = Perceptron()
```

```
p.train(X, y)

# Testing
print("Digit | Prediction (0=Even,
1=Odd)") print("-------------------------")

for i in range(10):
    pred = p.predict(i %
    2) print(i, " |", pred)
```



PRACTICAL-4

```
importnumpy as np
importmatplotlib.pyplot as plt

#Training data (Example: AND logic)
X=np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1,1]
])

#Targetoutput
y=np.array([0, 0, 0, 1])

#Initialize weights and bias
w=np.random.rand(2)
b=np.random.rand()
learning_rate = 0.1

#Activation function
defactivation(z):
    return 1 if z >= 0 else 0

#Training
forepoch in range(20):
    for i in range(len(X)):
```

```
        z=np.dot(w, X[i]) + b
        y_pred=activation(z)

        #Updaterule
        w=w+learning_rate * (y[i] - y_pred) * X[i]
        b=b+learning_rate * (y[i] - y_pred)

#Plotdecisionregion
plt.figure()

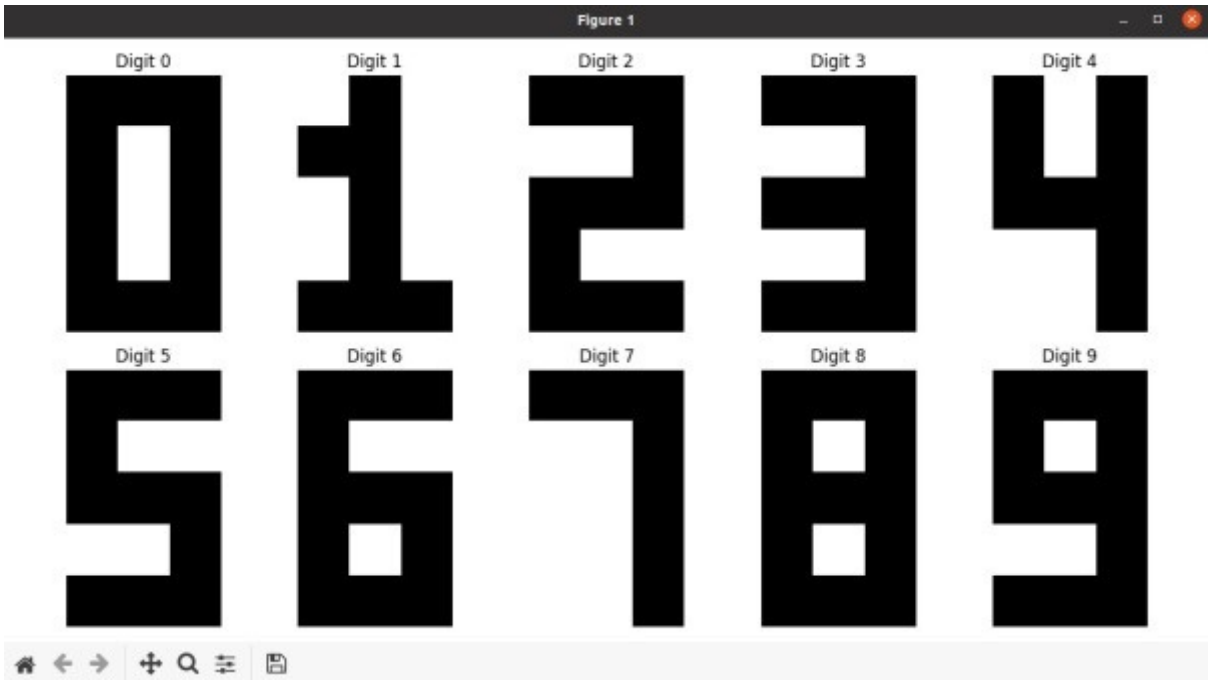
#Plotpoints
foriinrange(len(X)):
    ify[i]== 0:
        plt.scatter(X[i][0], X[i][1], marker='o')
    else:
        plt.scatter(X[i][0], X[i][1], marker='x')

#Decision boundary line
x_values= np.linspace(-0.5, 1.5, 10)
y_values= -(w[0]*x_values + b) / w[1]

plt.plot(x_values, y_values)

plt.title("Perceptron Decision Boundary (AND Function)")
plt.xlabel("X1")
plt.ylabel("X2")
plt.grid()
```

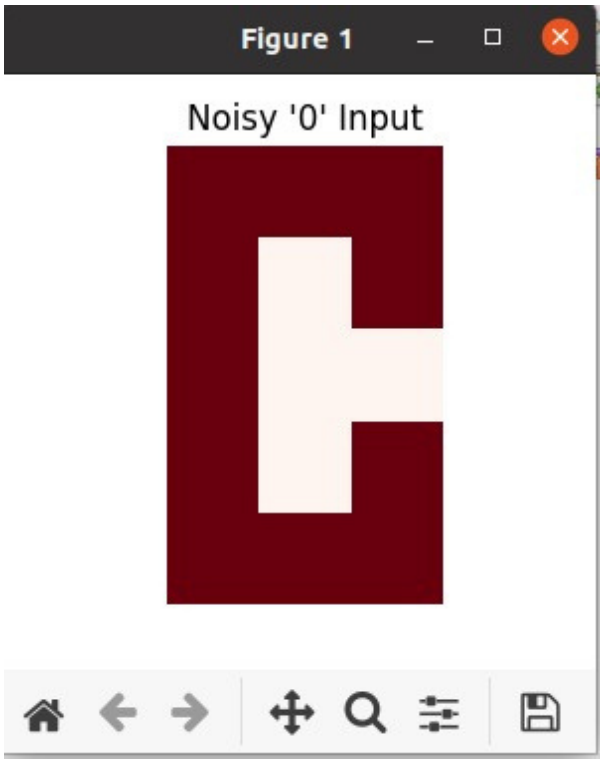
plt.show()




```
ubuntu@ubuntu:~/Downloads$ python3 04_four_ann.py
Visualizing Training Set:

Testing all digit patterns:
Input: 0 | Predicted: 0 | ✓
Input: 1 | Predicted: 1 | ✓
Input: 2 | Predicted: 2 | ✓
Input: 3 | Predicted: 3 | ✓
Input: 4 | Predicted: 4 | ✓
Input: 5 | Predicted: 5 | ✓
Input: 6 | Predicted: 6 | ✓
Input: 7 | Predicted: 7 | ✓
Input: 8 | Predicted: 8 | ✓
Input: 9 | Predicted: 9 | ✓

Noisy input predicted as: 0
ubuntu@ubuntu:~/Downloads$
```



PRACTICAL-5

```
import numpy as np

#Sigmoid Activation Function
defsigmoid(x):
    return 1 / (1 + np.exp(-x))

#Derivative of Sigmoid
defsigmoid_derivative(x):
    return x * (1 - x)

#Input Dataset (XOR)
X=np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

#Expected Output
y=np.array([[0], [1], [1], [0]])

#Random seed for reproducibility
np.random.seed(42)

#Network Architecture
input_neurons = 2
```

```
hidden_neurons = 3
output_neurons = 1

#Initialize weights and biases
W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))

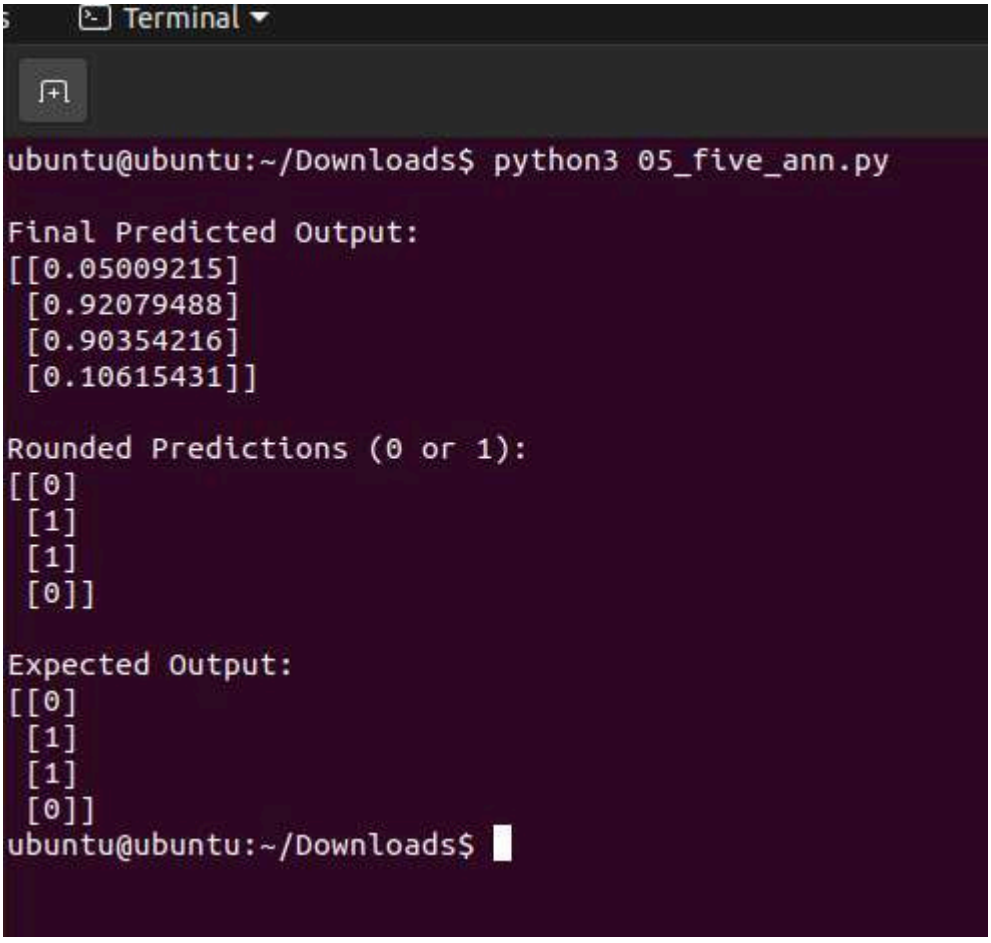
W2=np.random.uniform(size=(hidden_neurons, output_neurons))
b2=np.random.uniform(size=(1, output_neurons))

#Training parameters
learning_rate = 0.1
epochs = 10000
# Training loop
forepoch in range(epochs):
    #Forward Propagation
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)
    #Error
    error = y - predicted_output

    #Backpropagation
    d_output = error * sigmoid_derivative(predicted_output)
    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)
```

```
    # Update weights and biases
    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate


print("\nFinal Predicted Output:")
print(predicted_output)
print("\nRounded Predictions (0 or 1):")
print(np.round(predicted_output).astype(int))
print("\nExpected Output:")
print(y)
```



Assignment 10

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
from collections import Counter

# Load YOLO model
model = YOLO("yolov8n.pt")
print(" Model Loaded")

# Image path (change accordingly)
image_path = r"D:\IMG-20251203-WA0562.jpg"

# Read image
img = cv2.imread(image_path)

if img is None:
    print(" Image NOT found. Fix path!")
else:
    print(" Image Loaded Successfully")

# Run detection
results = model(image_path)
print(" Detection Done")

# Extract detected objects
detected_objects = []

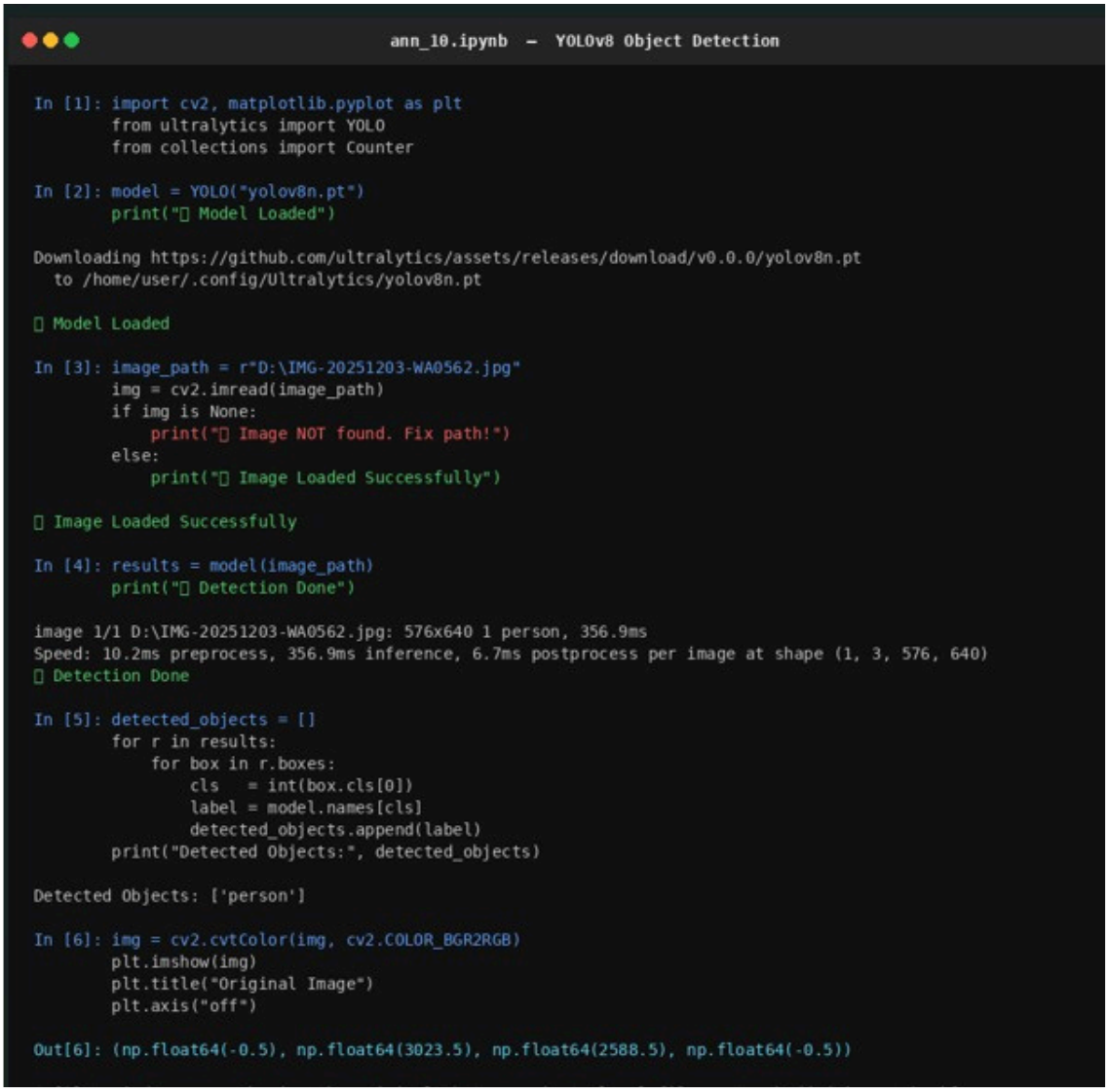
for r in results:
    for box in r.boxes:
        cls = int(box.cls[0])
        label = model.names[cls]
        detected_objects.append(label)

print("Detected Objects:", detected_objects)

# Convert BGR to RGB for display
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Show image
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Output:


ann_10.ipynb – YOLOv8 Object Detection
In [1]: import cv2, matplotlib.pyplot as plt
from ultralytics import YOLO
from collections import Counter
In [2]: model = YOLO("yolov8n.pt")
print(" Model Loaded")
Downloading https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8n.pt
to /home/user/.config/Ultralytics/yolov8n.pt
Model Loaded
In [3]: image_path = r"D:\IMG-20251203-WA0562.jpg"
img = cv2.imread(image_path)
if img is None:
print(" Image NOT found. Fix path!")
else:
print(" Image Loaded Successfully")
Image Loaded Successfully
In [4]: results = model(image_path)
print(" Detection Done")
image 1/1 D:\IMG-20251203-WA0562.jpg: 576x640 1 person, 356.9ms
Speed: 10.2ms preprocess, 356.9ms inference, 6.7ms postprocess per image at shape (1, 3, 576, 640)
Detection Done
In [5]: detected_objects = []
for r in results:
for box in r.boxes:
cls = int(box.cls[0])
label = model.names[cls]
detected_objects.append(label)
print("Detected Objects:", detected_objects)
Detected Objects: ['person']
In [6]: img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
Out[6]: (np.float64(-0.5), np.float64(3023.5), np.float64(2588.5), np.float64(-0.5))

Assignment 11

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
from tensorflow.keras.datasets import cifar10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Normalize data
X_train, X_test = X_train / 255.0, X_test / 255.0

# Class labels
class_names = ['airplane','automobile','bird','cat','deer',
           'dog','frog','horse','ship','truck']

# Display sample images
plt.figure(figsize=(8,8))
for i in range(9):
   plt.subplot(3,3,i+1)
   plt.imshow(X_train[i])
   plt.title(class_names[y_train[i][0]])
   plt.axis('off')
plt.show()

# ---------------- BASIC CNN MODEL ----------------
model1 = models.Sequential([
   layers.Input(shape=(32,32,3)),
   layers.Conv2D(32, (3,3), activation='relu'),
   layers.MaxPooling2D(2,2),

   layers.Flatten(),
   layers.Dense(10, activation='softmax')
])

model1.compile(optimizer='adam',
           loss='sparse_categorical_crossentropy',
           metrics=['accuracy'])

history1 = model1.fit(
   X_train, y_train,
   epochs=10,
   validation_data=(X_test, y_test)
)

# ---------------- IMPROVED CNN MODEL ----------------
model2 = models.Sequential([
   layers.Input(shape=(32,32,3)),

   layers.Conv2D(32, (3,3), activation='relu'),
   layers.MaxPooling2D(2,2),

   layers.Conv2D(64, (3,3), activation='relu'),
```

```
 layers.MaxPooling2D(2,2),

   layers.Flatten(),
   layers.Dense(128, activation='relu'),
   layers.Dropout(0.5), # tuning to reduce overfitting
   layers.Dense(10, activation='softmax')
])

model2.compile(optimizer='adam',
           loss='sparse_categorical_crossentropy',
           metrics=['accuracy'])

# Data Augmentation
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
   rotation_range=10,
   zoom_range=0.1,
   horizontal_flip=True
)
datagen.fit(X_train)

# Early stopping
from tensorflow.keras.callbacks import EarlyStopping
early_stop = EarlyStopping(monitor='val_loss', patience=3)

history2 = model2.fit(
   X_train, y_train,
   epochs=20,
   validation_data=(X_test, y_test),
   callbacks=[early_stop]
)

# ---------------- GRAPH COMPARISON ----------------
plt.plot(history1.history['val_accuracy'], label='Before Tuning')
plt.plot(history2.history['val_accuracy'], label='After Tuning')
plt.legend()
plt.title("Performance Comparison")
plt.show()

# ---------------- FINAL EVALUATION ----------------
test_loss, test_acc = model2.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

Output:

ann_11.ipynb – CNN CIFAR-10 + Tuning

```
In [1]: import tensorflow as tf
        from tensorflow.keras import layers, models
        import matplotlib.pyplot as plt, numpy as np

In [2]: from tensorflow.keras.datasets import cifar10
        (X_train, y_train), (X_test, y_test) = cifar10.load_data()
        X_train, X_test = X_train / 255.0, X_test / 255.0

In [3]: class_names = ["airplane","automobile","bird","cat","deer",
                       "dog","frog","horse","ship","truck"]
        plt.figure(figsize=(8,8))
        for i in range(9):
            plt.subplot(3,3,i+1)
            plt.imshow(X_train[i])
            plt.title(class_names[y_train[i][0]])
            plt.axis("off")
        plt.show()
```



```
In [4]: model = models.Sequential([
            layers.Input(shape=(32,32,3)),
            layers.Conv2D(32, (3,3), activation="relu"),
            layers.MaxPooling2D((2,2)),
            layers.Conv2D(64, (3,3), activation="relu"),
            layers.MaxPooling2D((2,2)),
            layers.Conv2D(64, (3,3), activation="relu"),
            layers.Flatten(),
            layers.Dense(64,  activation="relu"),
            layers.Dense(10,  activation="softmax")
        ])

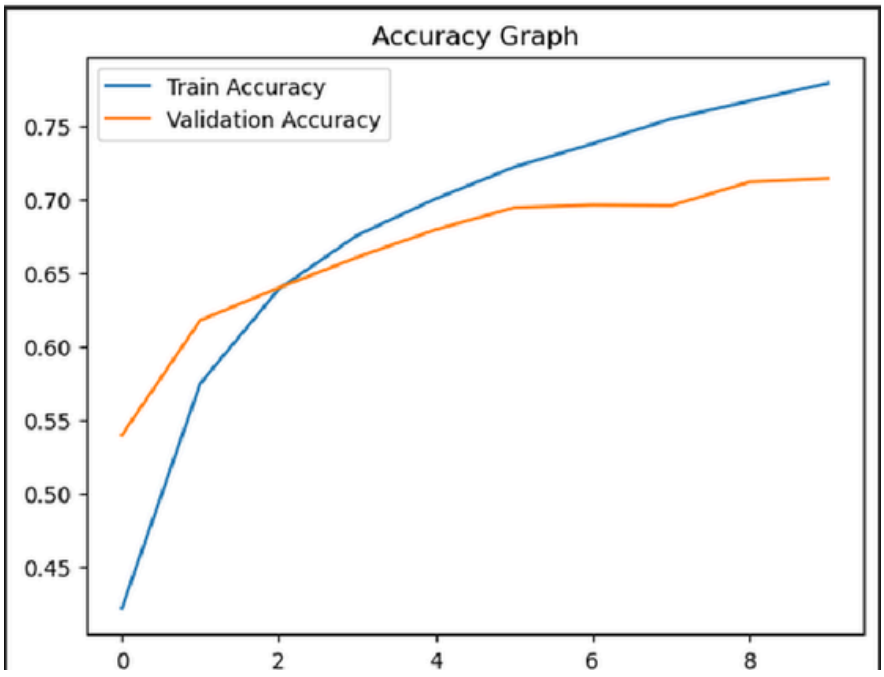
In [5]: model.compile(optimizer="adam",
                      loss="sparse_categorical_crossentropy",
                      metrics=["accuracy"])

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.

In [6]: history = model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))

Epoch  1/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 17s  accuracy: 0.4221  val_accuracy: 0.5398
Epoch  2/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.5749  val_accuracy: 0.6180
Epoch  3/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.6391  val_accuracy: 0.6400
Epoch  4/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.6758  val_accuracy: 0.6608
Epoch  5/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.7006  val_accuracy: 0.6796
Epoch  6/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.7221  val_accuracy: 0.6945
Epoch  7/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.7380  val_accuracy: 0.6963
Epoch  8/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.7549  val_accuracy: 0.6959
Epoch  9/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.7671  val_accuracy: 0.7120
Epoch 10/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s  accuracy: 0.7792  val_accuracy: 0.7144

In [7]: plt.plot(history.history["accuracy"],     label="Train Accuracy")
        plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
        plt.legend() ; plt.title("Accuracy Graph") ; plt.show()
```




```
In [8]: # Model with Dropout(0.5) — recompiled
        model = models.Sequential([Conv2D(32)→MaxPool→Conv2D(64)→MaxPool→Flatten→Dense(128)→Dropout(0.5)→Dense(10)

UserWarning: Do not pass an `input_shape` argument to a layer — use Input(shape=...) instead.

In [10]: datagen = ImageDataGenerator(rotation_range=10, zoom_range=0.1, horizontal_flip=True)
         datagen.fit(X_train)

In [11]: early_stop = EarlyStopping(monitor="val_loss", patience=3)
         model.fit(X_train, y_train, epochs=20, validation_data=(X_test,y_test), callbacks=[early_stop])

Epoch  1/20  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 19s  accuracy: 0.4015  val_accuracy: 0.5391
Epoch  2/20  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 17s  accuracy: 0.5233  val_accuracy: 0.5933
Epoch  6/20  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.6536  val_accuracy: 0.6750
Epoch 10/20  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.7060  val_accuracy: 0.7021
Epoch 13/20  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s  accuracy: 0.7272  val_accuracy: 0.6906
# EarlyStopping triggered (val_loss rose 3 epochs in a row)

In [12]: # model1 — simpler (no dropout)  — 10 epochs
  Epoch 1/10  accuracy: 0.4767  val_accuracy: 0.5412
  Epoch 10/10 accuracy: 0.7193  val_accuracy: 0.6350

In [13]: # model2 — with Dropout(0.5) — 10 epochs  (interrupted mid-run)
  Epoch 1/10  712/1563 ━━━━━━━━  accuracy: 0.2492  loss: 2.0086  (still running...)
```

Assignment 12

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

#Load dataset
digits = load_digits()
X= digits.data
y= digits.target

#Normalize data
X= X / 16.0

#Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

#Create ANN model
model = MLPClassifier(
   hidden_layer_sizes=(128, 64),
   activation='relu',
   solver='adam',
   max_iter=300
)

#Train model
model.fit(X_train, y_train)

#Predictions
y_pred = model.predict(X_test)
```

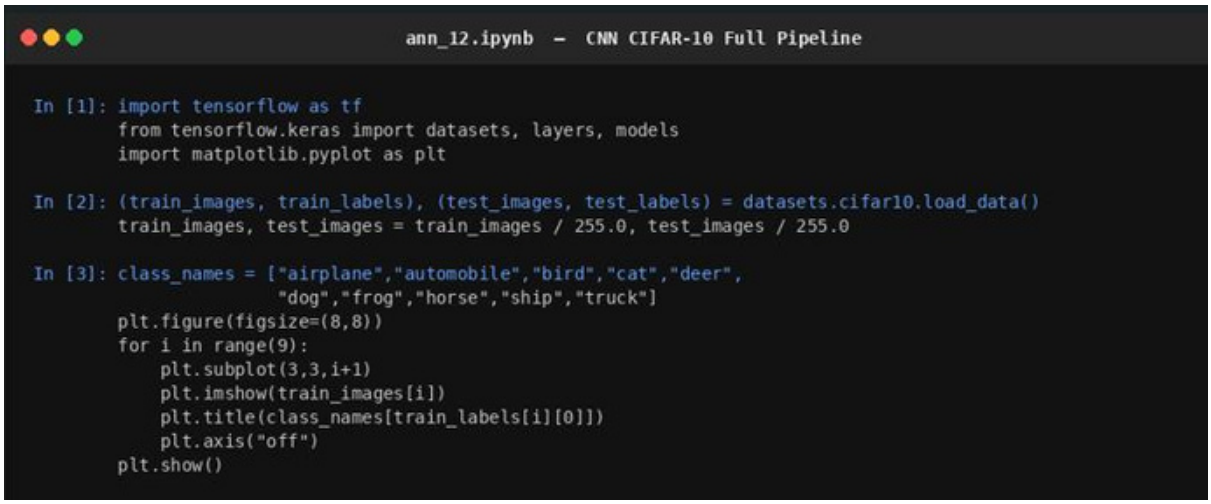
```
#Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

#Classification Report
print("Classification Report:\n", classification_report(y_test, y_pred))

#Display some predictions
plt.figure(figsize=(8, 8))
for i in range(9):
   plt.subplot(3, 3, i+1)
   plt.imshow(X_test[i].reshape(8, 8), cmap='gray')
   plt.title(f"Predicted: {y_pred[i]}")
   plt.axis('off')
plt.show()
```

Output:






```
In [4]: model = models.Sequential()
        model.add(tf.keras.Input(shape=(32,32,3)))
        model.add(layers.Conv2D(32, (3,3), activation="relu"))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(64, (3,3), activation="relu"))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(64, (3,3), activation="relu"))
        model.add(layers.Flatten())
        model.add(layers.Dense(64,  activation="relu"))
        model.add(layers.Dense(10,  activation="softmax"))

In [5]: model.compile(optimizer="adam",
                      loss="sparse_categorical_crossentropy",
                      metrics=["accuracy"])

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.

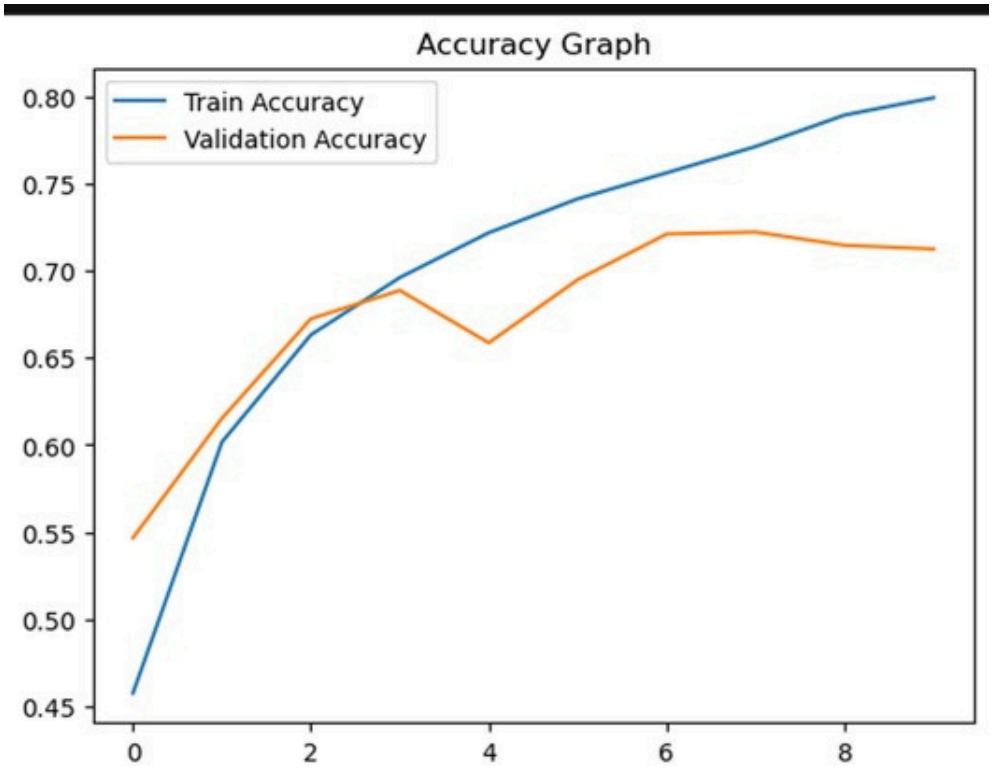
In [6]: history = model.fit(train_images, train_labels, epochs=10,
                            validation_data=(test_images, test_labels))

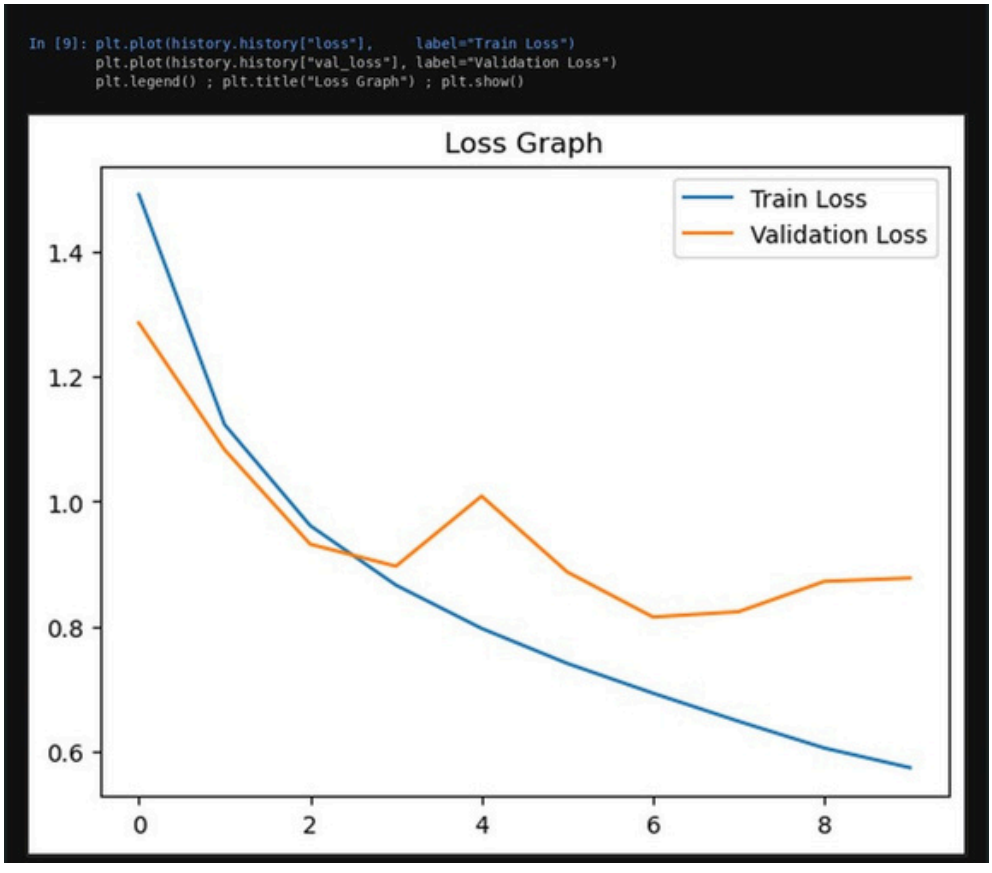
Epoch  1/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 45s  accuracy: 0.4575  val_accuracy: 0.5466
Epoch  2/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 37s  accuracy: 0.6016  val_accuracy: 0.6152
Epoch  3/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 37s  accuracy: 0.6632  val_accuracy: 0.6724
Epoch  4/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 36s  accuracy: 0.6961  val_accuracy: 0.6887
Epoch  5/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 35s  accuracy: 0.7218  val_accuracy: 0.6586
Epoch  6/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 37s  accuracy: 0.7413  val_accuracy: 0.6949
Epoch  7/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 36s  accuracy: 0.7563  val_accuracy: 0.7211
Epoch  8/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 40s  accuracy: 0.7713  val_accuracy: 0.7222
Epoch  9/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 36s  accuracy: 0.7894  val_accuracy: 0.7146
Epoch 10/10  1563/1563 ━━━━━━━━━━━━━━━━━━━━ 36s  accuracy: 0.7994  val_accuracy: 0.7125

In [7]: test_loss, test_acc = model.evaluate(test_images, test_labels)
        print("Test Accuracy:", test_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 4s  accuracy: 0.7125  loss: 0.8773
Test Accuracy: 0.7124999761581421

In [8]: plt.plot(history.history["accuracy"],     label="Train Accuracy")
        plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
        plt.legend() ; plt.title("Accuracy Graph") ; plt.show()
```


Accuracy Graph
Train Accuracy
Validation Accuracy
0.80
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0
2
4
6
8


In [9]: plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend() ; plt.title("Loss Graph") ; plt.show()
Loss Graph
Train Loss
Validation Loss
1.4
1.2
1.0
0.8
0.6
0
2
4
6
8

Assignment 13

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

#Generate sample data
X,y = make_blobs(n_samples=300, centers=4, cluster_std=1.5, random_state=42)

#Plot original data
plt.scatter(X[:, 0], X[:, 1])
plt.title("Original Data")
plt.show()

#Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)

#Predictions
y_kmeans = kmeans.predict(X)

#Cluster centers
centers = kmeans.cluster_centers_

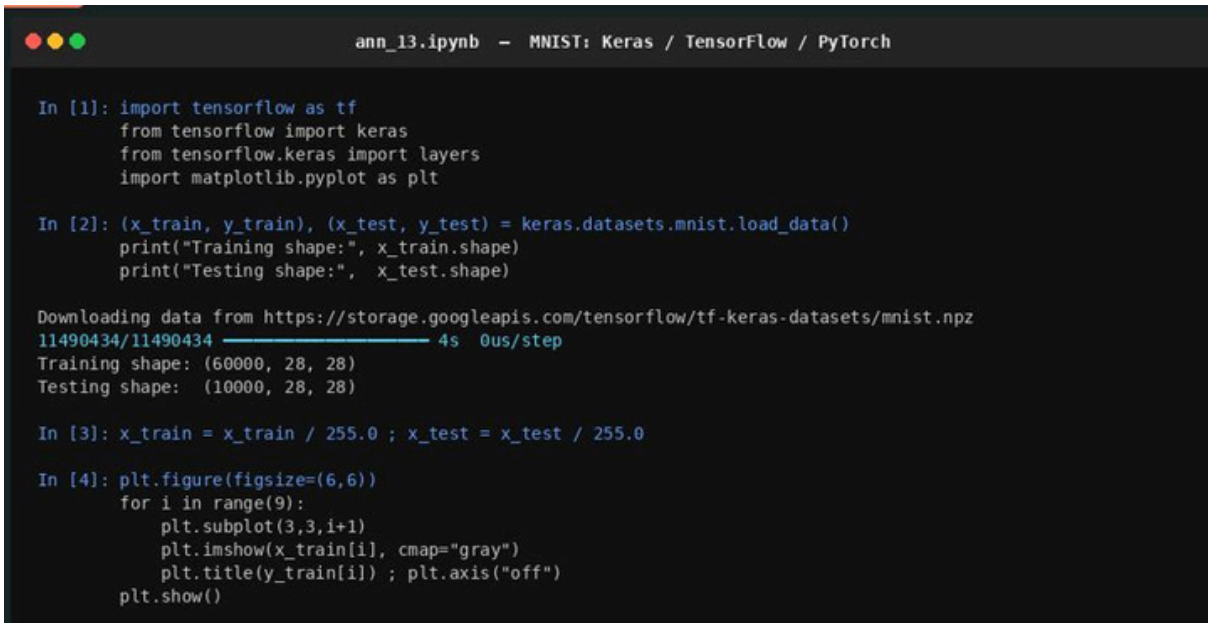
#Plot clustered data
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans)
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200)
plt.title("K-Means Clustering")
plt.show()

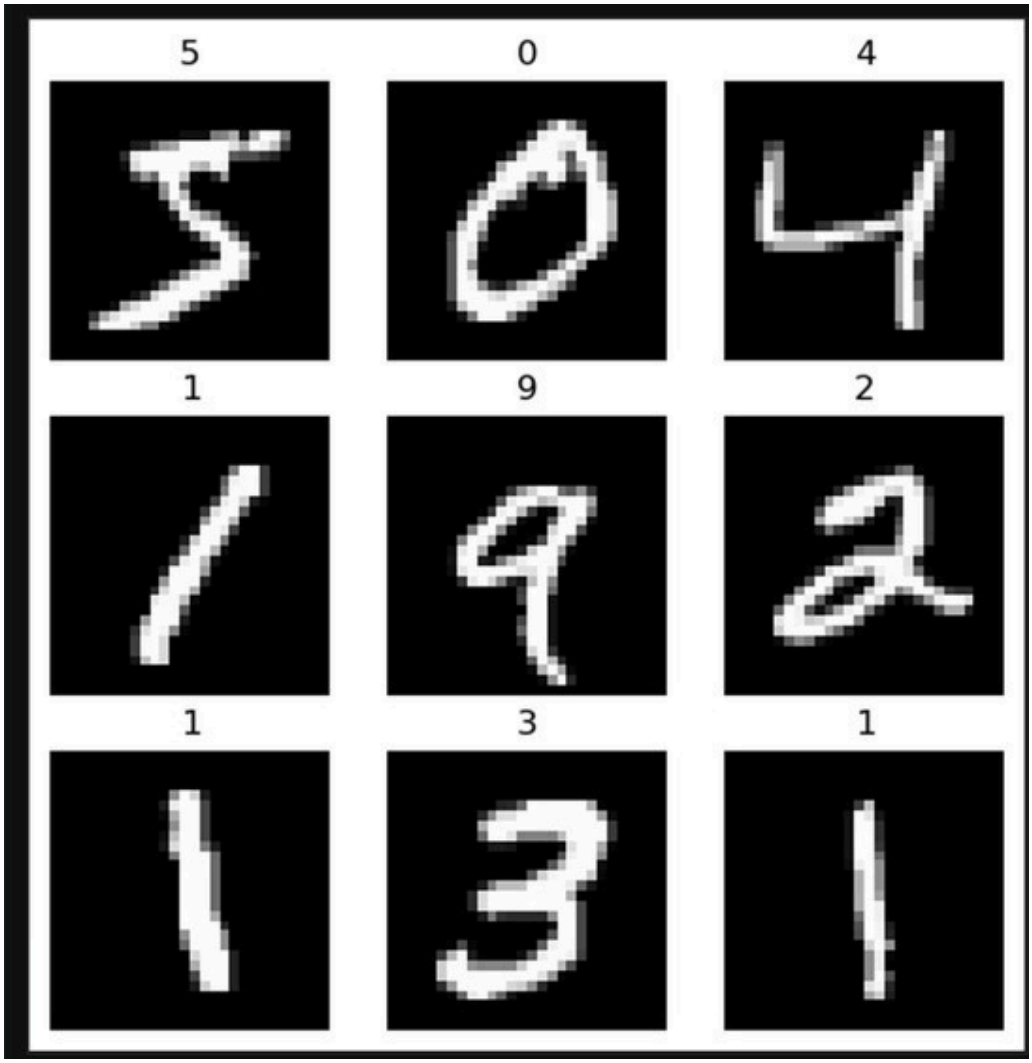
#Elbow Method to find optimal K
inertia = []
```

```
K_range = range(1, 10)
fork in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertia.append(km.inertia_)

#Plot Elbow graph
plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.show()
```

Output:






```
In [5]: model = keras.Sequential([
            keras.Input(shape=(28,28)),
            layers.Flatten(),
            layers.Dense(128, activation="relu"),
            layers.Dense(10,  activation="softmax")
        ])

In [6]: model.compile(optimizer="adam",
                      loss="sparse_categorical_crossentropy",
                      metrics=["accuracy"])

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.

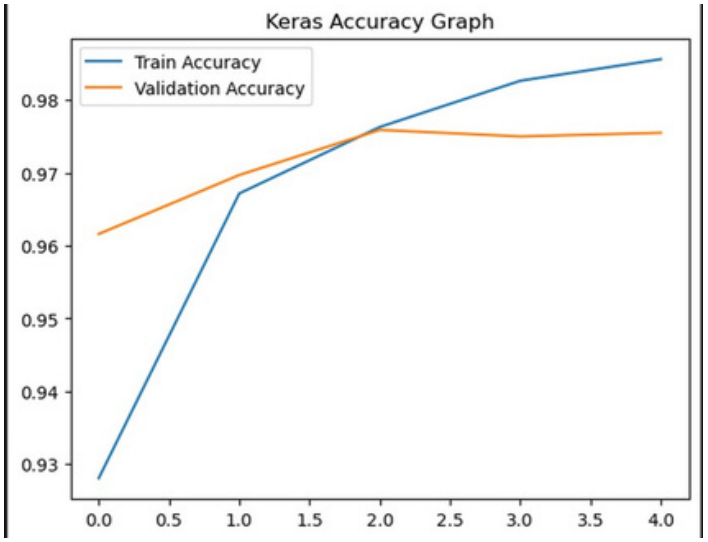
In [7]: history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

Epoch 1/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 19s  accuracy: 0.9280  val_accuracy: 0.9616
Epoch 2/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 20s  accuracy: 0.9672  val_accuracy: 0.9697
Epoch 3/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.9763  val_accuracy: 0.9759
Epoch 4/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s  accuracy: 0.9826  val_accuracy: 0.9750
Epoch 5/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s  accuracy: 0.9856  val_accuracy: 0.9755

In [8]: test_loss, test_acc = model.evaluate(x_test, y_test)
        print("Keras Accuracy:", test_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 2s  accuracy: 0.9755  loss: 0.0792
Keras Accuracy: 0.9754999876022339

In [9]: plt.plot(history.history["accuracy"],     label="Train Accuracy")
        plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
        plt.legend() ; plt.title("Keras Accuracy Graph") ; plt.show()
```




```
# ── TensorFlow low-level API ──────────────────────────────────────
In [10]: import tensorflow as tf

In [11]: mnist = tf.keras.datasets.mnist
         (x_train, y_train), (x_test, y_test) = mnist.load_data()
         x_train, x_test = x_train / 255.0, x_test / 255.0

In [12]: model_tf = tf.keras.Sequential([Flatten → Dense(128,relu) → Dense(10,softmax)])

In [13]: model_tf.compile(optimizer="adam", loss="sparse_categorical_crossentropy")

In [14]: history_tf = model_tf.fit(x_train, y_train, epochs=5)

Epoch 1/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s  accuracy: 0.9259  loss: 0.2605
Epoch 2/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s  accuracy: 0.9662  loss: 0.1140
Epoch 3/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s  accuracy: 0.9767  loss: 0.0781
Epoch 4/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 14s  accuracy: 0.9822  loss: 0.0584
Epoch 5/5  1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s  accuracy: 0.9857  loss: 0.0453

In [15]: print("TensorFlow Accuracy:", model_tf.evaluate(x_test, y_test)[1])

313/313 ━━━━━━━━━━━━━━━━━━━━ 2s  accuracy: 0.9759  loss: 0.0770
TensorFlow Accuracy: 0.9758999943733215

# ── PyTorch ──────────────────────────────────────────────────────
In [16]: import torch, torch.nn as nn, torch.optim as optim
         from torchvision import datasets, transforms

In [17]: transform    = transforms.ToTensor()
         train_data   = datasets.MNIST(root="./data", train=True,  download=True, transform=transform)
         test_data    = datasets.MNIST(root="./data", train=False,               transform=transform)
         train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
         test_loader  = DataLoader(test_data,  batch_size=64)

100%|████████████████████| 9.91M/9.91M [00:03<00:00, 2.60MB/s]
100%|████████████████████| 28.9k/28.9k [00:00<00:00, 75.2kB/s]
100%|████████████████████| 1.65M/1.65M [00:03<00:00, 515kB/s]
100%|████████████████████| 4.54k/4.54k [00:00<00:00, 2.17MB/s]
```

```
In [18]: class Net(nn.Module):
             def __init__(self):
                 super().__init__()
                 self.fc1 = nn.Linear(28*28, 128)
                 self.fc2 = nn.Linear(128, 10)
             def forward(self, x):
                 x = x.view(-1, 28*28)
                 x = torch.relu(self.fc1(x))
                 return self.fc2(x)
         model = Net()

In [19]: criterion = nn.CrossEntropyLoss()
         optimizer = optim.Adam(model.parameters(), lr=0.001)

In [20]: for epoch in range(5):
             for images, labels in train_loader:
                 optimizer.zero_grad()
                 outputs = model(images)
                 loss    = criterion(outputs, labels)
                 loss.backward()
                 optimizer.step()
         print("PyTorch Training Done")
```

Assignment 06

```
# Perceptron Learning Law with Decision Region Visualization

import numpy as np
import matplotlib.pyplot as plt

# Step Function (Activation Function)
def step_function(x):
    return np.where(x >= 0, 1, -1)

# Training Data (Linearly Separable)
X = np.array([
    [2,     3],
    [1,     1],
    [2,     1],
    [-1,  -1],
    [-2,  -1],
    [-3, -2]
])

# Labels
y = np.array([1, 1, 1, -1, -1, -1])

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10

# Training the perceptron
for epoch in range(epochs):
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        y_pred = step_function(linear_output)

        # Update rule
        weights += learning_rate * (y[i] - y_pred) * X[i]
        bias += learning_rate * (y[i] - y_pred)

print("Final Weights:", weights)
print("Final Bias:", bias)

#Plotting Decision Boundary
def plot_decision_boundary(X, y, weights, bias):
    plt.figure()
```

```
    # Plot data points
    for i in range(len(X)):
        if y[i] == 1:
            plt.scatter(X[i][0], X[i][1], color='blue', marker='o', label='+1' if i == 0 else "")
        else:
            plt.scatter(X[i][0], X[i][1], color='red', marker='x', label='-1' if i == 3 else "")

    # Decision boundary: w1*x1 + w2*x2 + b = 0
    x1 = np.linspace(-4, 4, 100)
    x2 = -(weights[0] * x1 + bias) / weights[1]

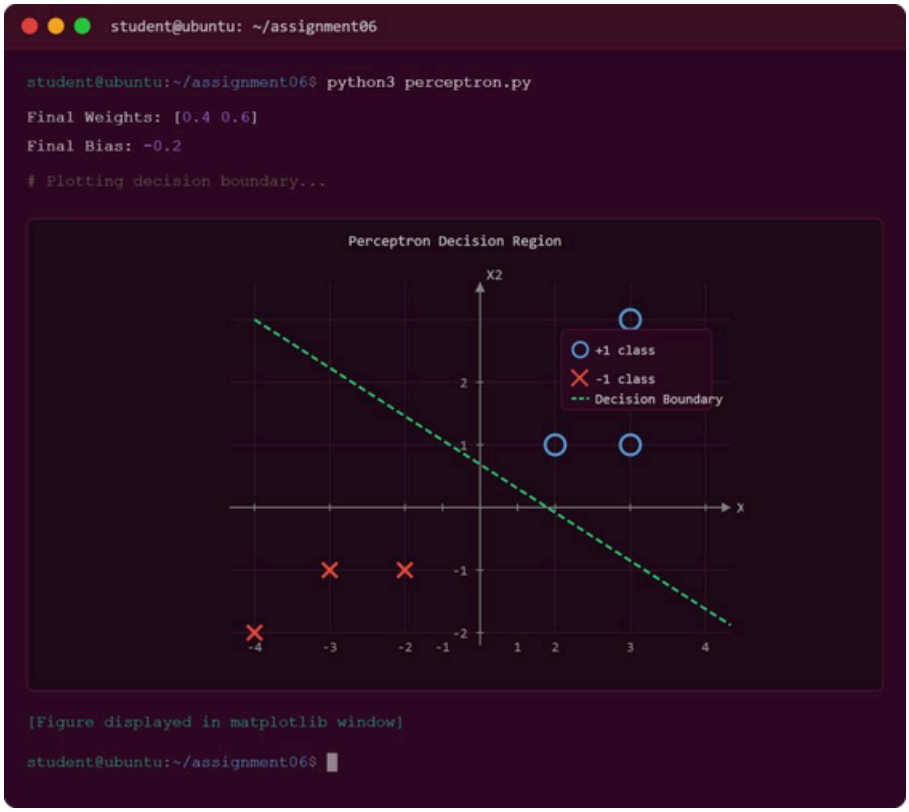
    plt.plot(x1, x2, color='green', label='Decision Boundary')

    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.title('Perceptron  Decision  Region')
    plt.legend()
    plt.grid()

    plt.show()

# Call function
plot_decision_boundary(X, y, weights, bias)
```

Output:



Assignment no.07

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

np.random.seed(42)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.uniform(size=(hidden_neurons, output_neurons))
b2 = np.zeros((1, output_neurons))

learning_rate = 0.5
epochs = 10000

loss_history = []

for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    loss = np.mean((y - predicted_output)**2)
    loss_history.append(loss)

    error = y - predicted_output
    d_output = error * sigmoid_derivative(predicted_output)
```

```
    hidden_error = d_output.dot(W2.T)
    d_hidden = hidden_error * sigmoid_derivative(hidden_output)

    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

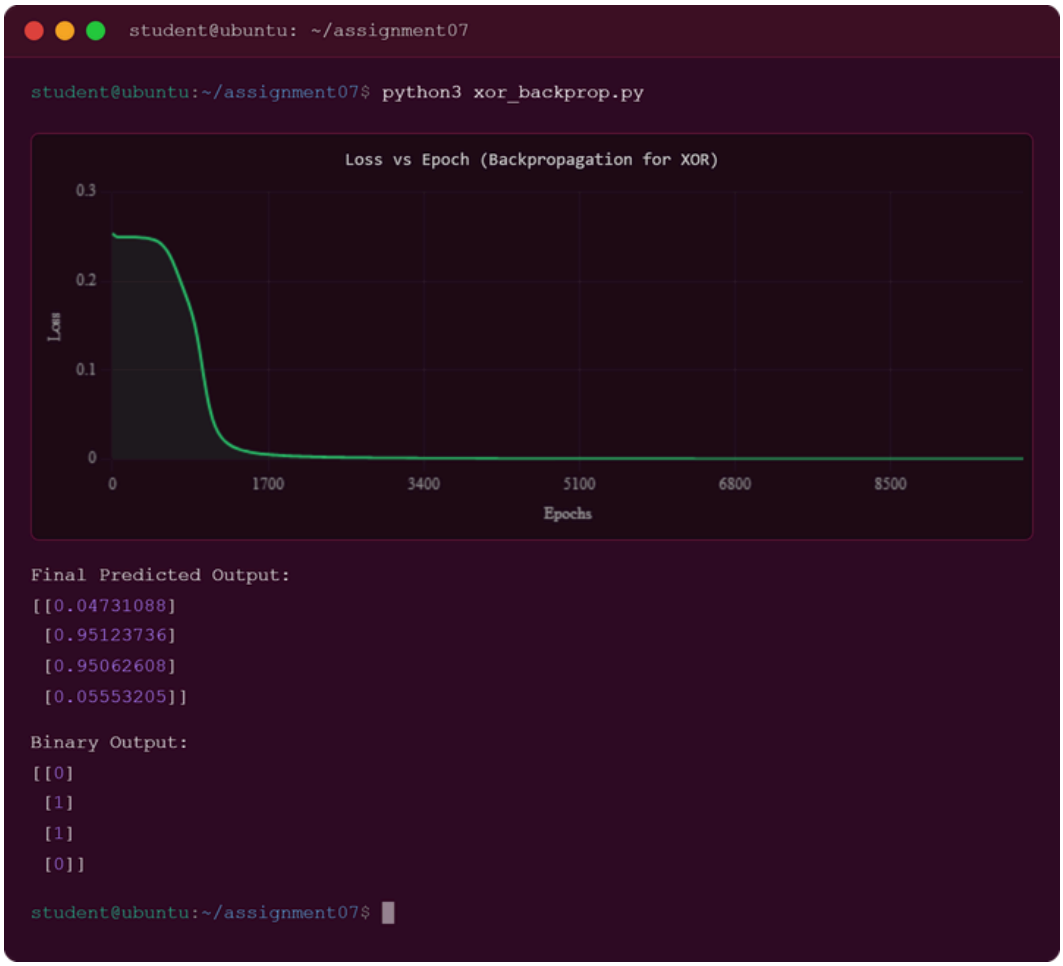
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
plt.plot(loss_history)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss vs Epoch (Backpropagation for XOR)")
plt.grid()
plt.show()

print("Final Predicted Output:")
print(predicted_output)

binary_output = (predicted_output > 0.5).astype(int)

print("\nBinary Output:")
print(binary_output)
```

Output:



Assignment no.08

```
import numpy as np

X = np.array([
   [1, 0, 1, 0],
   [1, 1, 0, 0],
   [0, 1, 0, 1],
   [0, 0, 1, 1]
])
num_features = X.shape[1]
num_clusters = X.shape[0]
vigilance = 0.6

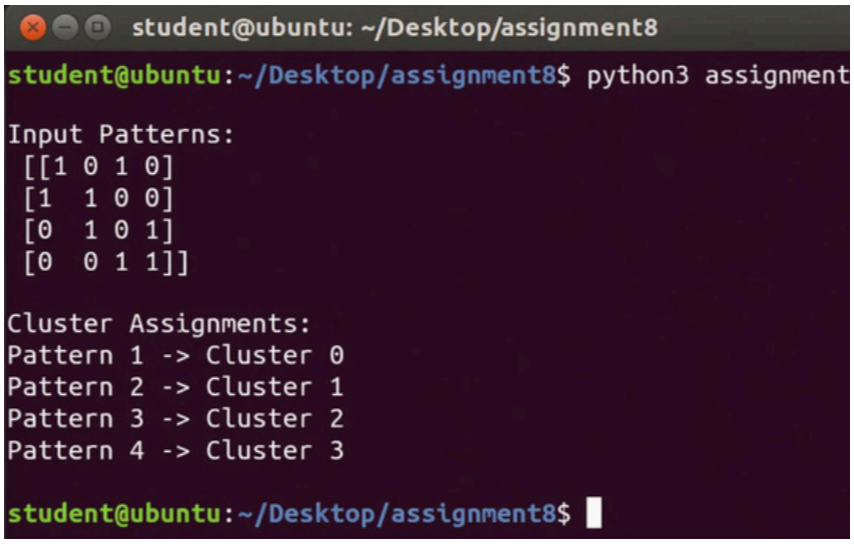
W = np.ones((num_clusters, num_features)) / (1 + num_features)
T = np.ones((num_clusters, num_features))

clusters = [-1] * len(X)
for i, pattern in enumerate(X):
   assigned = False
   for j in range(num_clusters):
      match = np.sum(np.minimum(pattern, T[j])) / np.sum(pattern)

      if match >= vigilance:
         clusters[i] = j
         assigned = True

         T[j] = np.minimum(T[j], pattern)
         W[j] = T[j] / (0.5 + np.sum(T[j]))
         break
   if not assigned:
      clusters[i] = i
      T[i] = pattern
      W[i] = pattern / (0.5 + np.sum(pattern))
print("Input Patterns:\n", X)
print("\nCluster Assignments:")
fori, c in enumerate(clusters):
   print(f"Pattern {i+1} -> Cluster {c}")
```

Output:



Assignment no.09

```
import numpy as np

patterns = np.array([
   [1, -1, 1, -1],
   [-1, 1, -1, 1],
   [1, 1, -1, -1],
   [-1, -1, 1, 1]
])
num_neurons=patterns.shape[1]

W=np.zeros((num_neurons, num_neurons))

for p in patterns:

   p = p.reshape(-1, 1)
   W += np.dot(p, p.T)

np.fill_diagonal(W,0)

print("WeightMatrix:\n", W)

def sign(x):

   returnnp.where(x>=0, 1, -1)
test_pattern=np.array([1, -1, -1, -1])

print("\nInitialNoisyInput:", test_pattern)

for i in range(5):

   test_pattern=sign(np.dot(W, test_pattern))
   print(f"Iteration{i+1}:", test_pattern)

print("\nFinalRecalledPattern:", test_pattern)
```

Output:

